

Scaffold 设计思路指导

本次比赛不考模型训练，也不要求你微调模型。你要做的是：**围绕一个通用大模型，设计一套更可靠的 SQL 生成流程。**

简单说，Scaffold 就是你写在大模型外面的程序逻辑。它负责决定：

给模型看什么信息
让模型先做什么、后做什么
什么时候执行 SQL
什么时候根据报错修正
什么时候返回最终 SQL

最终评测只看你输出的 SQL 执行结果是否正确，不看你的过程是否漂亮。但一个好的过程通常能显著提高正确率。

1. 不要只做“一次性生成 SQL”

最简单的做法是把题目、schema 全部塞进 prompt，让模型直接生成 SQL。这个 baseline 可以作为起点，但通常不够稳。

Text2SQL 容易错的原因很多：

模型选错表
漏掉 join 条件
字段名看起来像，但实际不是
自然语言里的值和数据库里的值不一致
SQL 语法能跑，但业务条件漏了
执行结果为空，模型却不知道哪里错了

更好的思路是把 Text2SQL 当作一个小型分析流程，而不是一次翻译：

理解问题
→ 找相关表字段
→ 必要时查看样本值
→ 生成 SQL
→ 执行 SQL
→ 根据结果或报错修正
→ 返回最终 SQL

APEX-SQL 的核心思想就是不要只相信静态 schema，而是让模型通过“假设—验证”的方式与数据库交互，逐步确认表、字段和值是否真的符合当前问题。

2. 先理解问题，再找表字段

建议你先让模型把问题拆成几个要素：

要查什么指标？
按什么维度分组？
有没有时间范围？
有没有筛选条件？
是否需要排序、Top-K、去重、比例计算？

例如：

问题：查询 2024 年每个渠道的订单 GMV Top 5。

可以拆成：
指标：GMV
时间：2024 年
维度：渠道
排序：GMV 从高到低
限制：Top 5

拆完以后，再去选择相关表和字段。不要一开始就把所有 schema 都丢给模型。schema 太多会干扰模型，很多表名和字段名都比较接近。

一个实用做法是：

先用关键词或简单规则筛出候选表字段
再让模型判断哪些表字段真正相关
最后只把相关 schema 放进 SQL 生成 prompt

如果不确定某个字段是不是必须的，宁可先保留。漏掉关键字段比多给几个无关字段更危险。

3. 遇到值不确定时，查一下真实数据

很多 Text2SQL 错误不是 SQL 写错，而是模型不知道数据库里的值怎么存。

例如：

用户说“加州”

数据库里可能是 'California', 也可能是 'CA'

用户说“已支付”

数据库里可能是 'paid', 也可能是 1, 或者 pay_status = 20

用户说“核心渠道”

数据库里可能是 SEM、FEED、BRAND

所以建议你的 scaffold 里加入一个很简单的 value probing 步骤：

```
SELECT DISTINCT column_name
FROM table_name
LIMIT 20;
```

或者：

```
SELECT column_name, COUNT(*)
FROM table_name
GROUP BY column_name
ORDER BY COUNT(*) DESC
LIMIT 20;
```

PV-SQL 的 Probe 思路就是这样：在生成最终 SQL 前，先用少量探查 SQL 看真实数据值，解决自然语言和数据库值之间的映射问题；例如确认“California”在库里是否存成“CA”。

这一步不需要复杂。你只需要在以下场景做探查：

自然语言里有地区、状态、渠道、类型、等级等枚举值

字段名看不出语义

SQL 需要写 WHERE 条件，但不知道具体取值

执行结果为空，怀疑过滤值写错

4. 生成 SQL 后，最好先执行一次

SQL 生成出来以后，不要马上返回。建议先执行一次，看是否：

语法错误
表不存在
字段不存在
join 写错
结果为空
结果列和问题不匹配

如果报错，把错误信息重新给模型，让它修正 SQL。

典型修复流程：

第 1 次生成 SQL
→ 执行
→ 报错：no such column
→ 把错误信息和 schema 再给模型
→ 让模型只修 SQL，不要重新发散
→ 再执行
→ 成功后返回

这种执行反馈很有价值。很多错误模型自己看不出来，但数据库一执行就会暴露。

5. 做几个简单的规则检查

即使 SQL 能执行，也不一定对。建议在返回前做一些轻量规则检查。

不用做复杂系统，先覆盖常见情况即可：

用户问“前 5 / top 5 / 最高的 5 个”
SQL 应该有 ORDER BY 和 LIMIT 5

用户问“多少 / 数量 / 个数”
SQL 通常应该有 COUNT

用户问“总额 / 合计”
SQL 通常应该有 SUM

用户问“平均”
SQL 通常应该有 AVG

用户问“比例 / 占比 / 转化率”
SQL 通常应该有除法，并注意分母为 0

用户问“唯一 / 不重复”

SQL 可能需要 DISTINCT 或 COUNT DISTINCT

用户问“2024 年 / 最近 30 天”

SQL 应该有时间过滤条件

PV-SQL 的 Verify 模块就是这个方向：从自然语言中抽取可检查的约束，再检查 SQL 是否漏掉这些显式要求。

在比赛里，你可以自己写很简单的正则或字符串检查。它不一定覆盖所有情况，但能抓住很多低级漏条件问题。

6. 可以生成多个候选 SQL，但不要盲目堆数量

对于简单题，一条 SQL 足够。对于复杂题，可以让模型生成 2-3 个候选 SQL，然后执行它们，选择更合理的结果。

候选选择可以很简单：

优先选择可执行的

优先选择结果非空的

优先选择满足规则检查的

优先选择使用了正确表字段的

优先选择没有多余列的

不要盲目生成很多候选。数量太多会浪费时间，也可能引入更多错误。

7. 比赛中不建议做的事情

不建议把所有 schema 一股脑塞进 prompt。schema 太多时，模型容易被无关字段干扰。

不建议只依赖模型自我反思。模型自己说“我检查过了”不等于真的对。最好用 SQL 执行结果和简单规则检查来验证。

不建议过度复杂化。两天时间有限，先把主流程跑稳，比做很多花哨模块更重要。

不建议硬编码测试集答案。隐藏评测会使用不同样本，硬编码没有意义，也违反比赛规则。

不建议让模型反复无限修正。设置最多 2-3 次修正即可，避免陷入循环。

8. 可以参考的论文思路

本次比赛只考 scaffold，不涉及训练。建议重点参考这些 **推理流程类** 思路：

APEX-SQL：核心思想是“不要只看 schema，要通过数据探查验证假设”。适合参考其 logical planning、schema pruning、data profiling、confirmation 的大方向。

PV-SQL：核心思想是 Probe + Verify。Probe 用来查看真实数据库值，Verify 用简单规则检查 SQL 是否漏掉用户问题中的硬条件。这个方法很适合两天比赛实现。

你不需要复现论文系统，也不需要实现复杂 Agent。比赛重点是：能否设计一个清晰、稳健、可执行的 SQL 生成Agent loop。

最重要的一句话：

不要把 Text2SQL 当成一次翻译任务，
要把它当成“生成 SQL → 执行验证 → 根据反馈修正”的Agent loop。